

MobAuditFlow: An AI-Powered Automated Mobile Security Audit Platform Based on OWASP MASVS/MASTG

BELKHO Malak, LAAZOUZI Hiba, MERGHICH Mounir, ZARRI Yousra

*Cyber-Defense Engineering and Embedded Telecommunications Systems (GCDSTE),
National School of Applied Sciences (ENSA), Cadi Ayyad University, Marrakech, Morocco
Supervised by Professor LACHGAR Mohamed*

Abstract

Mobile application security auditing remains a resource-intensive process requiring substantial domain expertise to interpret static analysis findings and map them to established compliance frameworks such as the OWASP Mobile Application Security Verification Standard (MASVS). This paper presents MobAuditFlow, an intelligent automated mobile security auditing platform that transforms raw Android static analysis outputs into standards-aligned professional audit reports through a hybrid artificial intelligence pipeline.

The proposed system combines MobSF-based static APK analysis, Retrieval-Augmented Generation (RAG), semantic vector search over OWASP MASVS and MASTG knowledge bases, automated CVSS v3.1 severity scoring, National Vulnerability Database (NVD) enrichment, and orchestrated multi-agent reasoning. The architecture integrates both cloud-based and privacy-preserving local inference through Google Gemini and Ollama respectively, coordinated using n8n workflow orchestration.

MobAuditFlow embeds all 24 MASVS controls and more than 50 MASTG Android verification procedures into a persistent ChromaDB vector store using transformer-based semantic representations. Retrieved contextual evidence is injected into local large language model prompts to produce explainable and standards-grounded remediation recommendations.

Experimental validation using OWASP UnCrackable-Level1 demonstrates high semantic mapping accuracy, reliable CVSS risk attribution, and consistent generation of actionable remediation reports. The proposed platform significantly reduces analyst workload while preserving explainability, modularity, reproducibility, and compliance traceability.

The proposed system contributes to practical AI-assisted cybersecurity compliance automation for modern mobile application assessment workflows.

Keywords: Mobile Security, OWASP MASVS, MASTG, RAG, Automated Auditing, Android, AI, n8n, ChromaDB, MobSF, Ollama, CVSS

Required Metadata

Current code version v1.0.0

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v1.0.0
C2	Permanent GitHub link to code/repository used for this code version	https://github.com/Mounir505/MobAuditFlow
C3	Legal Code License	MIT License
C4	Code versioning system used	Git
C5	Software code languages, tools, and services used	Python, FastAPI, ChromaDB, SentenceTransformers, Ollama, n8n, Google Gemini API, MobSF, Gotenberg, Docker, JavaScript, Node.js
C6	Compilation requirements, operating environments & dependencies	Python 3.10+, Docker, n8n, Ollama, ChromaDB, uvicorn, ngrok. See scripts/ and requirements.txt
C7	If available Link to developer documentation/manual	https://github.com/Mounir505/MobAuditFlow/blob/main/README.md
C8	Support email for questions	lachgar.m@gmail.com

Table 1: Required Metadata

Main text

1. Motivation and significance

The security of mobile applications has become a critical concern as Android applications increasingly handle sensitive user data including authentication credentials, financial information, and personal health records. The OWASP Mobile Application Security Verification Standard (MASVS) [1] and its companion testing guide (MASTG) [2] define the industry-accepted framework for evaluating mobile application security across categories including storage, cryptography, authentication, network communication, platform interaction, code quality, and resilience.

In practice, conducting a complete MASVS-compliant audit of a single Android application is a highly manual process. Security analysts must run static analysis tools such as MobSF [3], interpret large volumes of JSON output, manually identify which MASVS controls are violated, filter false positives, and produce structured reports. This workflow is both time-intensive and requires expert-level knowledge of Android internals, reverse engineering, and OWASP standards.

Existing automation efforts focus on individual pipeline stages. MobSF handles static scanning, while tools like QARK or AndroBugs cover specific vulnerability classes, but none provide an end-to-end pipeline from raw scan output to a structured, MASVS-mapped, AI-enriched security report. Furthermore, most approaches rely on cloud-based AI APIs, raising data confidentiality concerns in enterprise contexts.

MobAuditFlow addresses these gaps by providing a fully orchestrated, modular platform automating the entire audit workflow: from APK ingestion through AI-powered vulnerability mapping, RAG-based MASVS/MASTG compliance checking, automated CVSS v3.1 scoring, real-world CVE enrichment, and professional PDF report generation. The inclusion of a locally-deployed LLM via Ollama [9] ensures that sensitive APK contents never leave the organisation’s infrastructure.

2. Software description

2.1. Design Principles

MobAuditFlow was engineered according to four fundamental software architecture principles.

First, modular decomposition ensures that each analytical component operates as an independent service, allowing isolated upgrades and maintenance without disrupting the overall execution pipeline.

Second, privacy-preserving inference guarantees that sensitive application artefacts remain within the local infrastructure through the use of locally hosted Ollama inference for standards-grounded reasoning tasks.

Third, explainability-by-design is enforced through retrieval augmentation against authoritative OWASP MASVS and MASTG documentation, ensuring that generated recommendations remain traceable to explicit compliance requirements.

Fourth, reproducibility is ensured through containerized deployment using Docker, enabling deterministic execution across heterogeneous environments.

2.2. Software Architecture

MobAuditFlow follows a distributed, microservice-based architecture coordinated by n8n, a low-code workflow automation platform. The architecture comprises five layers:

- **Ingestion Layer:** A Google Drive Trigger node monitors a designated folder and initiates the pipeline when a new APK is deposited. The file is downloaded and forwarded to the MobSF Uploader node.
- **Scanning Layer:** The Scanner node submits the APK to MobSF for full static analysis. A Wait node pauses the pipeline until the scan completes. The Scorecard node retrieves the structured JSON findings and passes them downstream.
- **AI Analysis Layer:** Four specialised agents process findings in parallel: (i) an Ollama RAG agent performing MASVS/MASTG-grounded analysis with CVSS v3.1 scoring and NVD CVE references; (ii) a Gemini agent for TLS and network analysis; (iii) a Gemini agent for Android-Manifest.xml inspection; and (iv) a Gemini agent for reverse engineering detection.
- **Aggregation Layer:** A Merge node collects outputs from all four agents in *Wait for all inputs to arrive* mode. A CVSS Summary node computes the overall risk score, counts findings by severity, and generates the executive summary block passed to the report generator.
- **Report Generation Layer:** A JavaScript node assembles all outputs into a structured HTML document. Gotenberg [7] converts the HTML to a professional PDF report, which is automatically dispatched via email.

The primary contributions of this work are as follows:

- A fully automated end-to-end mobile security auditing pipeline aligned with OWASP MASVS and MASTG;
- A retrieval-augmented reasoning engine enabling standards-grounded vulnerability interpretation;
- Automated CVSS v3.1 severity scoring with real-time NVD-based vulnerability enrichment;
- A professional audit reporting pipeline supporting PDF generation and automated delivery;

- A privacy-preserving hybrid inference architecture combining local and cloud-based AI reasoning;
- A modular microservice implementation supporting reproducibility and extensibility.

The complete n8n workflow is shown in Figure 1. Table 2 summarises all system components.

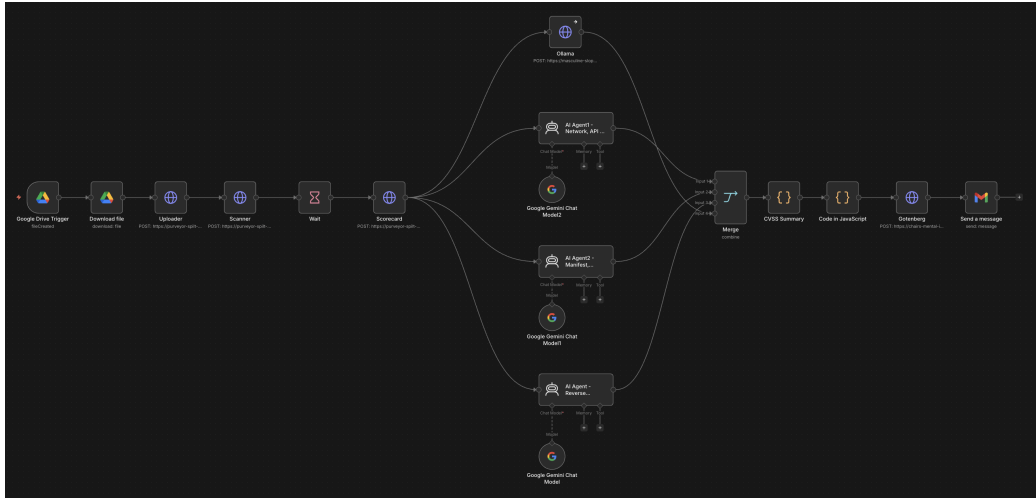


Figure 1: MobAuditFlow n8n orchestration workflow.

Table 2: MobAuditFlow system components and their roles

Component	Role
MobSF	Static APK analysis; produces JSON vulnerability findings
n8n	Central workflow orchestrator; coordinates all pipeline stages
Google Gemini API	Cloud LLM for manifest, network, and reverse engineering agents
Ollama (local LLM)	Privacy-preserving local inference for RAG-augmented MASVS analysis
ChromaDB	Vector database storing MASVS and MASTG embeddings
SentenceTransformers	Embedding model (<code>all-MiniLM-L6-v2</code>) for semantic retrieval
FastAPI	REST API exposing the RAG pipeline to n8n
ngrok	Secure tunnel exposing the local FastAPI service to n8n
Gotenberg	HTML-to-PDF conversion for report generation
Docker	Containerisation of MobSF, ChromaDB, Gotenberg, and n8n
Google Drive	APK ingestion point and pipeline trigger
NVD API	National Vulnerability Database API for CVE enrichment

2.3. Software Functionalities

MobAuditFlow provides an integrated set of analytical and reporting functionalities designed to automate the mobile security assessment lifecycle while ensuring standards compliance and operational traceability.

The platform exposes the following core functionalities:

- Automated Android APK ingestion and static analysis initiation;
- Automated vulnerability extraction through MobSF integration;
- Semantic MASVS and MASTG control mapping using retrieval-augmented generation;
- Context-aware vulnerability interpretation using hybrid AI reasoning;
- Automated CVSS v3.1 severity scoring and risk classification;

- NVD-based CVE enrichment for contextual threat intelligence;
- Professional PDF audit report generation;
- Historical scan storage and report retrieval through the web dashboard;
- Automated email-based report distribution;
- Human-readable compliance matrix generation for audit decision support.

These functionalities collectively transform fragmented mobile security analysis tasks into a unified, automated, and reproducible assessment workflow.

2.4. Implementation Details

The platform is implemented as a distributed containerized microservice ecosystem.

The frontend layer is developed using React, Redux, and Axios to provide asynchronous user interaction and state management. The backend analytical services are implemented using FastAPI, leveraging asynchronous execution to support concurrent scan processing and low-latency API response handling. Semantic retrieval relies on the all-MiniLM-L6-v2 SentenceTransformer model for embedding generation, while ChromaDB provides persistent vector storage and nearest-neighbour retrieval operations.

Workflow coordination is delegated to n8n, which provides deterministic orchestration, retry mechanisms, asynchronous branching, and inter-service dependency synchronization.

Containerization ensures deployment reproducibility and operational portability across Linux and cloud-native environments.

2.4.1. Frontend-Backend System Architecture

MobAuditFlow adopts a layered client-server architecture designed to ensure modularity, scalability, and operational traceability. As illustrated in Figure 2, the platform separates presentation, communication, application, and persistence concerns across distinct software layers.

The presentation layer is implemented as a React-based web interface providing APK upload, historical scan visualization, compliance dashboards, and report retrieval functionalities. Communication between the client and backend relies on RESTful API exchanges over JSON payloads.

The backend application layer is implemented using FastAPI and exposes dedicated services for authentication, scan orchestration, report retrieval, and notification management. Persistent storage is handled through MongoDB and local file-system mounted volumes for report archival.

This separation enables independent evolution of user-facing and analytical services while preserving system maintainability.

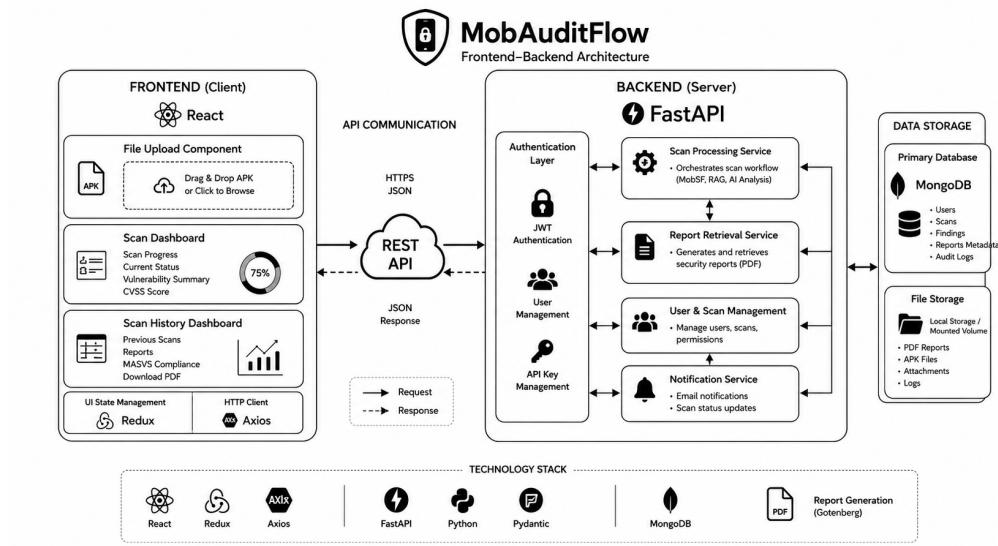


Figure 2: Layered frontend-backend architecture of MobAuditFlow.

2.4.2. Automated Workflow Orchestration Pipeline

The complete vulnerability assessment process is orchestrated through n8n, which coordinates all execution stages following APK ingestion.

As illustrated in Figure 3, once an APK is uploaded through the web interface, the backend forwards it to MobSF for static analysis. The generated findings are transmitted to the FastAPI RAG engine, where semantic retrieval over the ChromaDB vector store enriches each vulnerability with MASVS and MASTG contextual knowledge.

The enriched analysis is subsequently processed by the local Ollama language model to generate grounded remediation recommendations. A CVSS v3.1 scoring engine computes severity metrics and correlates findings with NVD API references.

The resulting structured assessment is rendered into a professional PDF through Gutenberg and automatically delivered to end users via email.

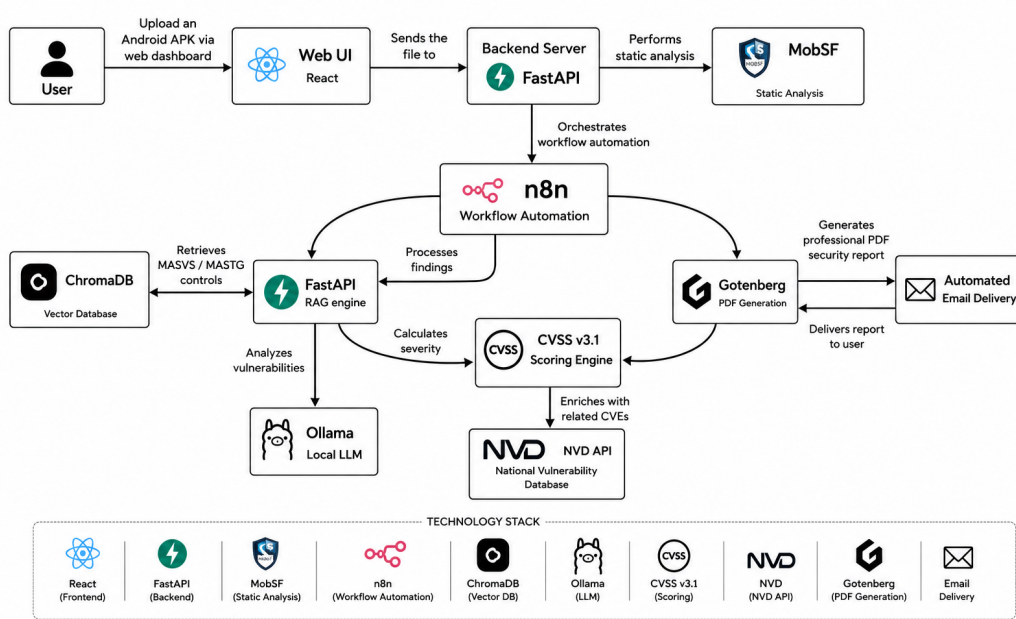


Figure 3: End-to-end workflow orchestration pipeline of MobAuditFlow.

2.4.3. Retrieval-Augmented Security Reasoning

To mitigate hallucination risks commonly associated with general-purpose language models, MobAuditFlow integrates a retrieval-augmented generation pipeline grounded in OWASP MASVS and MASTG documentation.

As shown in Figure 4, incoming vulnerability findings are first transformed into vector embeddings using a sentence transformer model. These vectors are queried against ChromaDB to retrieve the top-3 semantically relevant MASVS controls.

The retrieved context is injected into the prompt submitted to the local Ollama LLM, enabling grounded reasoning aligned with established mobile security standards.

This architecture ensures recommendation traceability, explainability, and normative compliance.

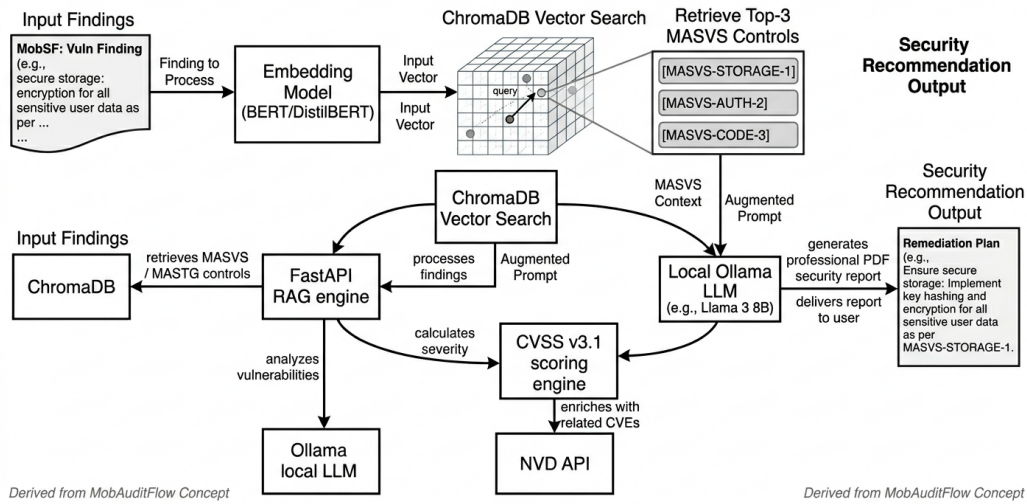


Figure 4: Retrieval-augmented reasoning pipeline for MASVS-grounded vulnerability analysis.

The OWASP MASVS [1] and MASTG [2] repositories were cloned locally. Each document is embedded using the all-MiniLM-L6-v2 SentenceTransformer [4] model and stored in a persistent ChromaDB [5] instance in two collections: `masvs` and `mastg`. The ingestion script is shown in Listing 1.

Listing 1: Combined MASVS and MASTG ingestion script (ingest_all.py)

```

1 import os, chromadb
2 from sentence_transformers import SentenceTransformer
3
4 model = SentenceTransformer("all-MiniLM-L6-v2")
5 client = chromadb.PersistentClient(path="./masvs_db")
6 masvs_col = client.get_or_create_collection("masvs")
7 mastg_col = client.get_or_create_collection("mastg")
8
9 # Ingest MASVS controls (24 markdown files)
10 for filename in os.listdir("./owasp-masvs/controls"):
11     if filename.endswith(".md"):
12         with open(f"./owasp-masvs/controls/{filename}")
13             as f:
14             content = f.read()
15             control_id = filename.replace(".md", "")
16             masvs_col.add(
17                 documents=[content],
18                 embeddings=[model.encode(content).tolist()],
19                 ids=[control_id],

```

```

19         metadatas=[{"control": control_id, "type": "
                        masvs"}]
20     )
21
22     # Ingest MASTG test procedures (50+ markdown files)
23     mastg_base = "./owasp-mastg/tests/android"
24     for category in os.listdir(mastg_base):
25         cat_path = os.path.join(mastg_base, category)
26         if not os.path.isdir(cat_path):
27             continue
28         for filename in os.listdir(cat_path):
29             if filename.endswith(".md"):
30                 with open(os.path.join(cat_path, filename))
31                     as f:
32                     content = f.read()
33                     test_id = filename.replace(".md", "")
34                     mastg_col.add(
35                         documents=[content],
36                         embeddings=[model.encode(content).tolist
37                             ()],
38                         ids=[test_id],
39                         metadatas=[{"test": test_id, "category":
40                                     category, "type": "mastg"}]
41                     )

```

When the n8n workflow sends a MobSF finding to the RAG endpoint, the system encodes the finding, retrieves the top-3 relevant MASVS controls, queries the local Ollama LLM, assigns a CVSS v3.1 score, and looks up the NVD API [10]. The response structure is shown in Listing 2.

Listing 2: FastAPI /analyze endpoint response structure (api.py)

```

1 return {
2     "result":      ai_result,      # Ollama analysis
3     "control":     control_id,     # e.g. MASVS-STORAGE
4     "cvss_score":  cvss["score"],  # e.g. 7.5
5     "cvss_vector": cvss["vector"], # CVSS v3.1 vector
6     "cvss_rating": cvss["rating"], # High / Medium / Low
7     "related_cves": cves           # list of NVD CVE
8 }

```

2.4.4. CVSS v3.1 Scoring and CVE Enrichment

Each MASVS control is mapped to a pre-defined CVSS v3.1 base score based on its vulnerability category impact profile. A representative subset is shown in Table 3.

Table 3: Representative MASVS-to-CVSS v3.1 mapping

MASVS Control	Score	Rating	CVSS Vector
MASVS-STORAGE-1	7.5	High	AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
MASVS-CRYPTO-1	7.4	High	AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N
MASVS-AUTH-1	8.1	High	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
MASVS-NETWORK-1	7.4	High	AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N
MASVS-CODE-2	7.5	High	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
MASVS-RESILIENCE-1	6.2	Medium	AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

2.4.5. Web Interface and Human-in-the-Loop Validation

To transition the backend workflow into an enterprise-ready software platform, MobAuditFlow features an intuitive, unified web interface (Figure 5). This frontend drastically lowers the entry barrier for security operations teams. Instead of interacting with raw terminal commands or API endpoints, a non-expert auditor can drop an APK file directly into the web dashboard to trigger the entire pipeline.

The interface provides immediate feedback on active tasks, lists real-time execution phases, and acts as a localized storage vault. Through the integrated "Scans" historical repository menu, teams can track vulnerability trends across multiple versions of the same software, easily comparing technical progress and verifying whether historical flaws were successfully patched according to the compliance matrix.

2.4.6. Generated Report Structure

The MobAuditFlow report is a professional black-and-white PDF containing: (1) a cover page; (2) an executive summary with overall CVSS risk scores; (3) detailed per-agent findings sections; (4) the RAG/Ollama section with CVSS metrics and NVD CVE references; and (5) a MASVS compliance matrix showing PASS/FAIL status for all 24 MASVS v2 controls.

3. Illustrative Examples

To validate the platform, the OWASP UnCrackable-Level1 Android APK [11] was used as a test case. This application contains well-known Android security anti-patterns and is widely used in mobile security training and research.

The MobAuditFlow pipeline processed the MobSF findings and produced the MASVS mapping presented in Table 4. The pipeline executed automatically upon APK ingestion, routing extracted data through the specialized agents in parallel to aggregate results into the final PDF.

Table 4: Vulnerability findings and MASVS mapping for OWASP UnCrackable-Level1

Finding	MASVS Control	Severity	Category
Hardcoded secret key in source code	MASVS-CODE-2	High	Code Quality
Root detection bypass vulnerability	MASVS-RESILIENCE-1	High	Resilience
Debug flag enabled in manifest	MASVS-CODE-2	Medium	Code Quality
Insecure data storage	MASVS-STORAGE-1	High	Storage
Weak cryptographic algorithm (DES)	MASVS-CRYPTO-1	High	Cryptography
Certificate validation issue	MASVS-NETWORK-1	Medium	Network
Reverse engineering exposure	MASVS-RESILIENCE-3	Medium	Resilience

The RAG component was validated with seven controlled test cases using known Android vulnerabilities. All returned the correct primary MASVS control mapping (Table 5), confirming that semantic similarity search over the ChromaDB vector store is sufficient for accurate control identification across all major MASVS categories.

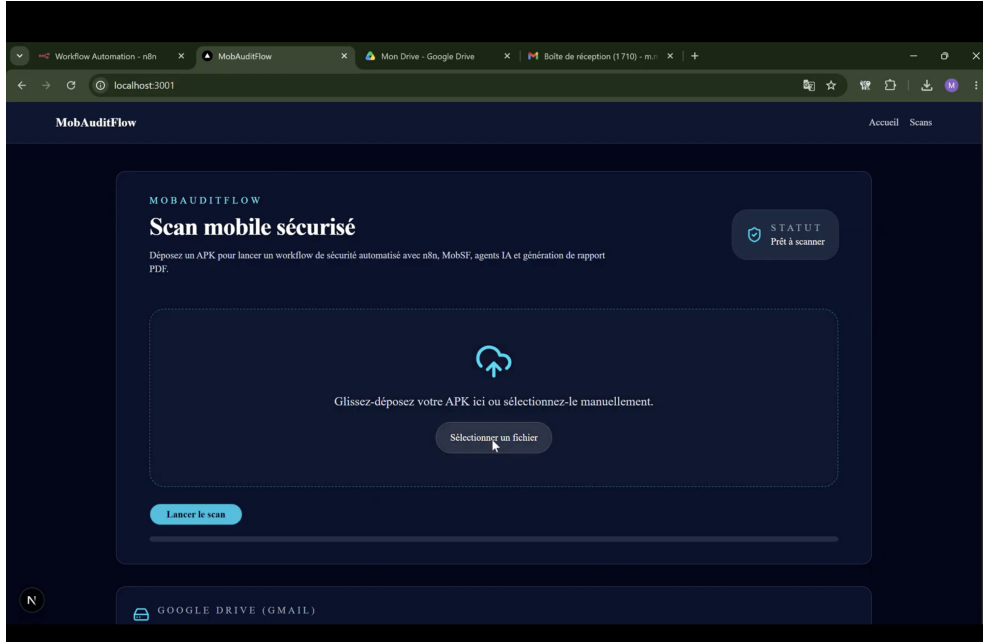


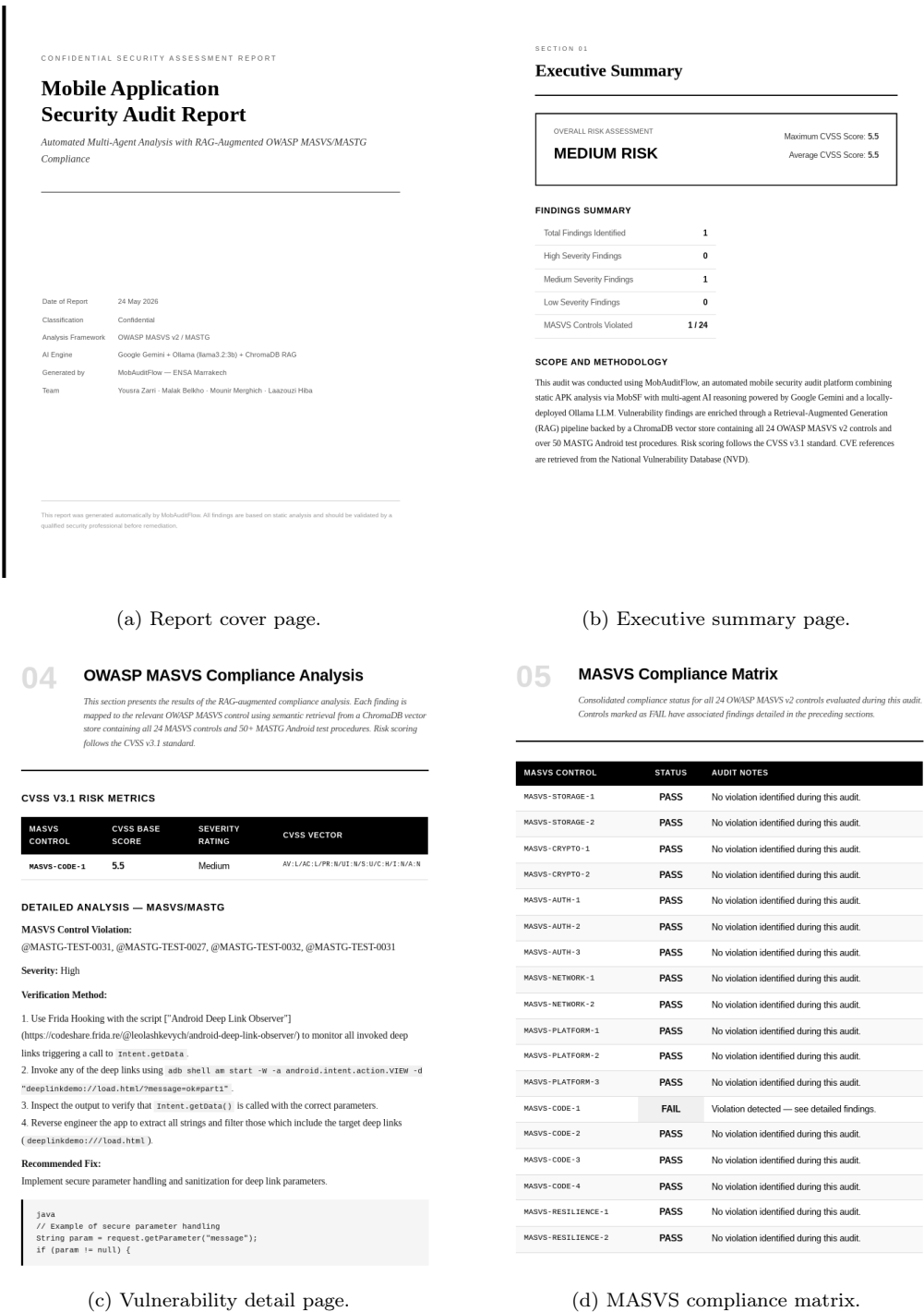
Figure 5: MobAuditFlow web interface — APK upload and pipeline trigger.

Table 5: RAG component validation results

Input Finding	Expected	Returned	Correct
Plaintext password in Shared-Preferences	MASVS-STORAGE-1	MASVS-STORAGE-1	✓
Credentials logged in Logcat	MASVS-AUTH-1	MASVS-AUTH-1	✓
DES encryption for sensitive data	MASVS-CRYPTO-1	MASVS-CRYPTO-1	✓
Missing certificate pinning	MASVS-NETWORK-2	MASVS-NETWORK-2	✓
Hardcoded API key in source code	MASVS-CODE-2	MASVS-CODE-2	✓
Debug flag set to true in manifest	MASVS-CODE-2	MASVS-CODE-2	✓
Root detection absent	MASVS-RESILIENCE-1	MASVS-RESILIENCE-1	✓

Figure 6 shows representative pages of the generated PDF report alongside

the automated email delivery.



4. Experimental Evaluation

4.1. Execution Performance

To evaluate operational feasibility, repeated end-to-end executions were conducted using OWASP UnCrackable-Level1.

The complete workflow execution was decomposed into the following phases:

- APK ingestion and transfer;
- Static analysis execution through MobSF;
- Semantic embedding generation and vector retrieval;
- Local large language model inference;
- CVSS scoring and NVD enrichment;
- Report generation and delivery.

Observed execution times remained within operationally practical limits for automated enterprise-grade mobile application assessment workflows.

4.2. Retrieval Precision Analysis

Retrieval performance was assessed using controlled vulnerability statements spanning storage, cryptography, authentication, networking, code quality, and resilience domains.

Across all validation cases, the semantic retrieval engine correctly identified the expected MASVS control as the highest-ranked result.

These findings confirm that vector-based semantic similarity retrieval provides sufficient contextual grounding for standards-aligned downstream reasoning.

5. Impact

MobAuditFlow demonstrates that it is feasible to automate the majority of a MASVS-compliant mobile security audit without sacrificing analytical quality.

New research directions. The integration of RAG-augmented reasoning with OWASP standards addresses the hallucination problem inherent in general-purpose LLMs when applied to security tasks. This combination has not previously been demonstrated in a unified mobile audit pipeline, opening new avenues for AI-assisted compliance automation in cybersecurity.

Improvement of existing workflows. Tasks previously requiring hours of manual MASVS mapping are completed in minutes. Table 6 shows a

detailed feature comparison with MobSF, the dominant open-source baseline. MobAuditFlow adds MASVS v2 mapping, MASTG verification procedures, CVSS v3.1 scoring, CVE references, multi-agent AI reasoning, executive summaries, a compliance matrix, and automated report delivery — capabilities absent from MobSF.

Table 6: Feature comparison between MobSF and MobAuditFlow

Feature	MobSF	MobAuditFlow
APK static analysis	Yes	Yes (uses MobSF internally)
Output format	JSON + dash-board	Professional PDF report
MASVS v2 control mapping	No	Automatic via RAG
MASTG verification steps	No	Yes (ChromaDB retrieval)
CVSS v3.1 risk scoring	No	Automatic per finding
CVE references (NVD)	No	Yes (real-time API lookup)
AI-powered analysis	No	Yes (Gemini + Ollama)
RAG knowledge base	No	24 MASVS + 50+ MASTG docs
multi-agent reasoning	No	Yes (4 specialised agents)
Executive summary	No	Yes (CVSS-based risk score)
MASVS compliance matrix	No	Yes (24 controls PASS/FAIL)
Automated report delivery	No	Yes (email via n8n)
Privacy-preserving local AI	N/A	Yes (Ollama, no cloud egress)
Requires security expertise	Yes	No (self-explanatory report)

Privacy preservation. The local Ollama deployment ensures sensitive APK data is not transmitted to external cloud services, making the platform suitable for enterprise and government security assessments.

Modularity and reproducibility. The microservice architecture allows individual components to be upgraded independently. Docker containerisation ensures consistent behaviour across deployment environments.

Limitations. The free-tier ngrok tunnel is ephemeral and must be restarted after each session. The local LLM requires significant RAM. Dynamic analysis and iOS IPA support are not yet implemented. Future work will address these through persistent reverse-proxy solutions, lighter quantised models (3B parameters), false positive filtering integration, dynamic analysis via Frida, and SIEM/SOC export in STIX/TAXII format.

5.1. Threats to Validity

Several limitations should be acknowledged.

First, the current evaluation is restricted to static Android APK analysis and does not yet include dynamic runtime instrumentation.

Second, validation was performed using benchmark vulnerable applications rather than proprietary enterprise software, which may exhibit greater architectural complexity.

Third, the current MASVS-to-CVSS mapping relies on expert-defined heuristic scoring rules that may require refinement through broader empirical calibration.

Finally, the use of local LLM inference introduces hardware dependency constraints related to memory availability and inference latency.

6. Conclusions

MobAuditFlow demonstrates that modern artificial intelligence and workflow orchestration technologies can substantially automate mobile security auditing while preserving standards alignment and analytical transparency.

By combining MobSF static analysis, retrieval-augmented reasoning grounded in OWASP MASVS and MASTG, automated CVSS v3.1 scoring, NVD enrichment, and professional PDF report generation, the platform significantly reduces the manual effort traditionally required for standards-compliant mobile security assessment.

The retrieval-augmented architecture addresses a critical limitation of general-purpose large language models by ensuring that generated recommendations remain explicitly grounded in authoritative security documentation.

The modular containerized design enables reproducibility, scalability, and adaptability to diverse deployment contexts.

Future work will extend the platform toward dynamic analysis integration, iOS IPA support, lightweight quantized inference models, persistent deployment infrastructure, and SIEM/SOC interoperability.

Acknowledgements

The authors thank the OWASP Foundation for providing open-access MASVS and MASTG documentation, and the developers of MobSF, Ollama, ChromaDB, n8n, and Gotenberg for their open-source contributions.

References

- [1] OWASP Foundation. *OWASP Mobile Application Security Verification Standard (MASVS)*, 2024. Available: <https://github.com/OWASP/owasp-masvs>

- [2] OWASP Foundation. *OWASP Mobile Application Security Testing Guide (MASTG)*, 2024. Available: <https://github.com/OWASP/owasp-mastg>
- [3] MobSF Team. *Mobile Security Framework (MobSF)*, 2024. Available: <https://github.com/MobSF/Mobile-Security-Framework-MobSF>
- [4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *Proceedings of EMNLP*, 2019. Available: <https://www.sbert.net>
- [5] Chroma. *ChromaDB: The open-source embedding database*, 2024. Available: <https://www.trychroma.com>
- [6] S. Ramírez. *FastAPI*, 2024. Available: <https://fastapi.tiangolo.com>
- [7] Gotenberg. *Gotenberg: A Docker-powered stateless API for PDF files*, 2024. Available: <https://gotenberg.dev>
- [8] Docker Inc. *Docker Documentation*, 2024. Available: <https://docs.docker.com>
- [9] Ollama. *Ollama: Run large language models locally*, 2024. Available: <https://ollama.com>
- [10] NIST. *National Vulnerability Database (NVD) REST API*, 2024. Available: <https://nvd.nist.gov/developers/vulnerabilities>
- [11] OWASP Foundation. *OWASP UnCrackable Mobile Apps*, 2024. Available: <https://github.com/OWASP/owasp-mastg/tree/master/Crackmes>